



Intel® Data Center GPU Verification Test Suite

User Guide

April 2026

Revision 0.6

Intel Confidential



Notice: This document contains information on products in the design phase of development. The information here is subject to change without notice. Do not finalize a design with this information.

Intel technologies may require enabled hardware, software or service activation.

You may not use or facilitate the use of this document in connection with any infringement or other legal analysis concerning Intel products described herein. You agree to grant Intel a non-exclusive, royalty-free license to any patent claim thereafter drafted which includes subject matter disclosed herein.

No license (express or implied, by estoppel or otherwise) to any intellectual property rights is granted by this document.

The products described may contain design defects or errors known as errata which may cause the product to deviate from published specifications. Current characterized errata are available on request.

Performance results are based on testing as of dates shown in configurations and may not reflect all publicly available updates. See backup for configuration details. No product or component can be absolutely secure.

Performance varies by use, configuration, and other factors. Learn more on the [Performance Index site](#).

Your costs and results may vary.

"Conflict-free" refers to products, suppliers, supply chains, smelters, and refiners that, based on our due diligence, do not contain or source tantalum, tin, tungsten or gold (referred to as "conflict minerals" by the U.S. Securities and Exchange Commission) that directly or indirectly finance or benefit armed groups in the Democratic Republic of the Congo or adjoining countries.

All product plans and roadmaps are subject to change without notice.

Code names are used by Intel to identify products, technologies, or services that are in development and not publicly available. These are not "commercial" names and not intended to function as trademarks.

Intel disclaims all express and implied warranties, including without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement, as well as any warranty arising from course of performance, course of dealing, or usage in trade.

Results have been estimated or simulated.

Intel does not control or audit third-party data. You should consult other sources to evaluate accuracy.

Intel contributes to the development of benchmarks by participating in, sponsoring, and/or contributing technical support to various benchmarking groups, including the BenchmarkXPRT Development Community administered by Principled Technologies.

Copies of documents which have an order number and are referenced in this document may be obtained by calling 1-800-548-4725 or visiting the [Resource and Documentation Center](#).

© 2026 Intel Corporation. Intel, the Intel logo, Arc, and other Intel marks are trademarks of Intel Corporation or its subsidiaries. Other names and brands may be claimed as the property of others.

Contents

1	Introduction	7
1.1	High Level Test Descriptions	7
1.2	Reference.....	8
1.3	Terminology	8
2	Host System and Test Setup.....	10
2.1	Live GPU Monitoring (X11 Forwarding User Guide).....	10
2.1.1	Using SSH from Linux* or macOS*	10
2.1.2	Using PuTTY on Windows*	11
2.1.3	Enable Live Usage Monitoring	12
3	Test Execution.....	13
3.1	Menu Test Execution.....	13
3.1.1	Reset Tests	14
3.1.2	Power and Thermal Stress Test	15
3.1.3	Excursion Design Power Stress Test.....	15
3.1.4	vLLM Benchmark Test.....	16
3.1.5	ML Perf Benchmark Test.....	17
3.2	CMD Line Test Execution	20
3.2.1	GPU Health Check	21
3.2.2	GPU Environment and Device Check	21
3.2.3	PCIe Bandwidth Test.....	21
3.2.4	PCIe Lane Margin Test	22
3.2.5	Reset Test.....	22
3.2.6	Memory Bandwidth Test.....	23
3.2.7	Memory Stress Test.....	23
3.2.8	Power and Thermal Stress Test.....	23
3.2.9	Excursion Desing Power Stress Test.....	24
3.2.10	Functional Test	24
3.2.11	vLLM Test.....	24
3.2.12	ML Perf Test	26
3.2.13	OneCCL Collective Test	26
3.2.14	OneCCL Point to Point Test	26
3.2.15	Run All.....	27
3.2.16	Debug Script	27
3.2.17	Collect Logs.....	27
3.3	JSON Config File Execution	27
4	Theory of Operations.....	29
4.1	GPU Health Check	29
4.1.1	Low Level Test Description	29
4.1.2	Pass or Fail Criteria	29
4.2	GPU Environment and Device Check	29
4.2.1	Low Level Test Description	29
4.2.2	Pass or Fail Criteria	29
4.3	PCIe Bandwidth Test.....	30
4.3.1	Low Level Test Description	30
4.3.2	Pass or Fail Criteria	30

4.4	PCIe Lane Margin Test	31
4.4.1	Low Level Test Description	31
4.4.2	Pass or Fail Criteria	31
4.5	Reset Tests	32
4.5.1	Setup	32
4.5.2	Execution Flow.....	32
4.5.3	Use	32
4.5.4	Results and Pass/Fail Criteria.....	35
4.6	Memory Bandwidth Test.....	35
4.6.1	Low Level Test Description	35
4.6.2	Pass or Fail Criteria	35
4.7	Memory Stress Test.....	36
4.7.1	Low Level Test Description	36
4.7.2	Pass or Fail Criteria	36
4.8	Power and Thermal Stress Test.....	37
4.8.1	Low Level Test Description	37
4.8.2	Pass or Fail Criteria	37
4.9	Excursion Design Power Stress Test.....	37
4.9.1	Low Level Test Description	37
4.9.2	Pass or Fail Criteria	38
4.10	Functional Test	38
4.11	vLLM Test.....	38
4.11.1	Low Level Test Description	38
4.11.2	Configuration and Sanity Checks	39
4.11.3	Docker Preparation.....	39
4.11.4	Container Lifecycle	39
4.11.5	Server Startup and Readiness.....	39
4.11.6	Benchmark Execution	40
4.11.7	Cleanup	40
4.11.8	Interpreting Results.....	40
4.12	MLPerf*	41
4.12.1	Low Level Test Description	41
4.12.2	Pass or Fail Criteria	42
4.13	OneCCL Collective Test	42
4.14	One CCL Point to Point Test	43
4.15	Debug Script	43
4.16	HTML Summary Report	45
5	Issues and Workarounds.....	47
5.1	IOMMU Can Cause ACS to Reenable or Fail to Disable.....	47
5.2	Running Some Card Tests While Running XPUM Can Cause a Crash or Bus Fatal Error.....	47
5.3	Intel® DG Diagnostic Tool Tests Only Run on Devices Located on PCIe Segment 0	47
5.4	PCIe Lane Margin Tool (LMT) Can Cause PCIe Fatal Bus Errors and Fails to Finish	47
5.5	PCIe BW Test Bi-Directional memory_benchmark May Fail	48
5.6	Power, Thermal, and Excursion Design Power Stress (EDP) Tests May Not Work with Intel® Arc™ Pro B70 GPU Cards	48

5.7	GPU Environment and Device Check Memory Bandwidth Tests May Fail due to Lower Expected Memory Bandwidth	48
5.8	Monitoring and CPU stress tests do not work with vLLM benchmark ..	48
6	Troubleshooting	49
6.1	Live Monitoring Windows Not Showing	49
6.1.1	X11 Forwarding Not Enabled.....	49
6.1.2	DISPLAY Variable Not Set.....	49
6.1.3	X Server Not Running on Client.....	49
6.1.4	Manual X11 Test	49
6.1.5	Root Permission and Xauthority Issues	49
6.1.6	Mixed SSH Sessions	49
6.1.7	Restricted Environments	49
6.2	vLLM benchmarking.....	50

Figures

Figure 2-1.	Putty Configuration	11
Figure 2-2.	GPU Usage Monitoring	12
Figure 3-1.	Tests Menu	14
Figure 3-2.	Access to the Models.....	18
Figure 3-2.	Access Token for Model Download.....	19
Figure 4-1.	Example InfoCollect Result Summary Output	43
Figure 4-2.	Summary Report Example	45
Figure 4-3.	Test Details Section Example	46

Tables

Table 1-1.	Reference Documents.....	8
Table 1-2.	Terms and Definitions.....	8
Table 3-1.	PCIe Bandwidth Tests Subset	22

Revision History

Revision Number	Description	Date
0.6	- Added details for vLLM and MLPerf* tests in Section 1.1. - Added Section 2.1 about Live GPU Monitoring. - Added description for System pre-flight checks in Section 3. - Updated Section 3.1.1 with new reset types enabled. - Added Section 3.1.4 about vLLM benchmark test. - Added Section 3.1.5 about MLPerf test. - Updated Section 3.2 with new parameters introduced on VTS 0.6. - Added Section 4.11 vLLM. - Added Section 4.12 MLPerf test. - Added Section 6 Troubleshooting. - Updated Table 3-1. - Modified pass/fail criteria for Power and Thermal and EDP Stress Tests in Sections 4.8.2 and 4.9.2.	April 2026
0.5	Initial release of the document.	February 2026

1 Introduction

This document describes the Intel® Data Center GPU Verification Test Suite intended for testing and validating system and accelerator functionality during hardware integration into supported server configurations.

This release was tested on and supports Intel® Arc™ Pro B70 graphics Software Development Vehicles using the latest drivers from <https://dgpu-docs.intel.com/driver/client/overview.html>.

1.1 High Level Test Descriptions

- **GPU Health Check:** Checks the GPU core and memory temperature, GPU power, memory health, and GPU frequency.
- **GPU Environment and Device Check:** Checks environment variables, libraries, permissions, device software exclusivity, PCI Express* (PCIe*), computation check, computation performance, media codec, power, memory bandwidth, memory allocation, memory errors, and memory bandwidth.
- **PCIe Bandwidth Test:** Measures PCIe bandwidth and compares it against expected value.
- **PCIe Lane Margin Test (LMT):** Measures PCIe eye margin at GPU Rx and compares it against expected value.
- **Reset Test:** Tests device enumeration reliability, link training, secondary bus reset, and function level reset functionality and host-level reset stability.
- **Memory Bandwidth Test:** Measures memory bandwidth and compares it against expected value.
- **Memory Stress Test:** Stresses memory performing various operations and ensures no ECC errors are detected.
- **Power and Thermal Stress Test:** Stresses the CPU and GPUs at TDP for given time and checks if measured average power and maximum temperature are as expected.
- **Excursion Design Power Stress Test:** Generates stress pulses on the GPU for a given time while maintaining a constant CPU stress. Checks if the GPU can reach TDP on those pulse cycles.
- Functional Test (Not Available Yet)
- **vLLM Test:** vLLM benchmark testing is the performance evaluation of the vLLM inference engine. The testing measures key metrics including throughput (requests/tokens per second), latency (time to first token and response times), memory usage efficiency.

- **MLPerf* Test:** MLPerf Inference is the industry-standard benchmark suite designed by the MLCommons consortium to measure how quickly Machine Learning (ML) systems can run trained models to make predictions from new data.
- OneCCL Collective Test (Not Available Yet)
- OneCCL Point to Point Test (Not Available Yet)
- **Run All:** Runs all the preselected tests with a single command.
- **Debug Script:** The `infoCollector.sh` script checks the general system status, setup, and functionality with a focus on GPUs. Generates log files for debug analysis.
- **Collect Logs:** Collects all generated test logs, generates a summary HTML report and compresses everything into a zip file.

1.2 Reference

See the following documents for additional information:

Table 1-1. Reference Documents

Document Title	Document Number or Location
<i>Intel Battlemage GPU Installation Guide</i>	https://dgpu-docs.intel.com/driver/client/overview.html
<i>SDV for Intel® Crescent Island Platform Setup Guide</i>	873451

1.3 Terminology

The following terms are used in this document:

Table 1-2. Terms and Definitions

Term	Description
CAD	Computer-Aided Design
DMA	Direct Memory Access
ECC	Error Correction Code
EDP	Excursion Design Power
FEC	Forward Error Correction
FLR	Function-Level Reset
GPU	Graphics Processing Unit
HBM	High-Bandwidth Memory
MME	Matrix Multiplication Engine
NIC	Network Interface Card
NVMe-MI*	Non-Volatile Memory Express* Management Interface

OSFP	Octal Small Form Factor Pluggable
PCIe*	Peripheral Component Interconnect Express*
PID	Process ID
QSFP	Quad Small Form Factor Pluggable
SBR	Secondary Bus Reset
SER	Symbol Error Rate
Test wrapper	Single interface to start verification scripts
TPC	Tensor Processor Core
VTS	Verification Test Suite

2 *Host System and Test Setup*

This section describes how to configure the host system and provides the installation prerequisites for the verification tests.

For Intel® Data Center GPUs, the Intel® Data Center GPU Verification Test Suite (VTS) requires Ubuntu* 25.10 with the latest Intel® Data Center GPU firmware and driver. To install the latest firmware and drivers, go to the following link and complete the setup steps:

<https://dgpu-docs.intel.com/driver/client/overview.html>

Additional Host Requirements

- Intel® Arc™ Pro B70 GPU

Downloading and Unzipping of the Intel® Data Center GPU Verification Test Suite

1. Download the Intel® Data Center GPU Verification Test Suite from the Resource and Documentation Center (RDC).
2. Copy the Intel® Data Center GPU Verification Test Suite to the system under test.
3. Unzip the Intel® Data Center GPU Verification Test Suite:

```
sudo unzip  
<873489_Intel_Data_Center_GPU_Verification_Test_Suite_Rev_0_6>  
.zip
```

Other requirements

All Linux* libraries, python packages, and tools required will be automatically checked and installed by the VTS during runtime.

2.1 **Live GPU Monitoring (X11 Forwarding User Guide)**

To view live GPU monitoring using the **-live_mon** option, the host system must be able to display graphical output on your client system. This requires X11 forwarding to be enabled on your SSH connection.

2.1.1 **Using SSH from Linux* or macOS***

When connecting from a Linux or macOS client, use X11 forwarding (-Y):

```
ssh -Y <username>@<hostsystem>
```

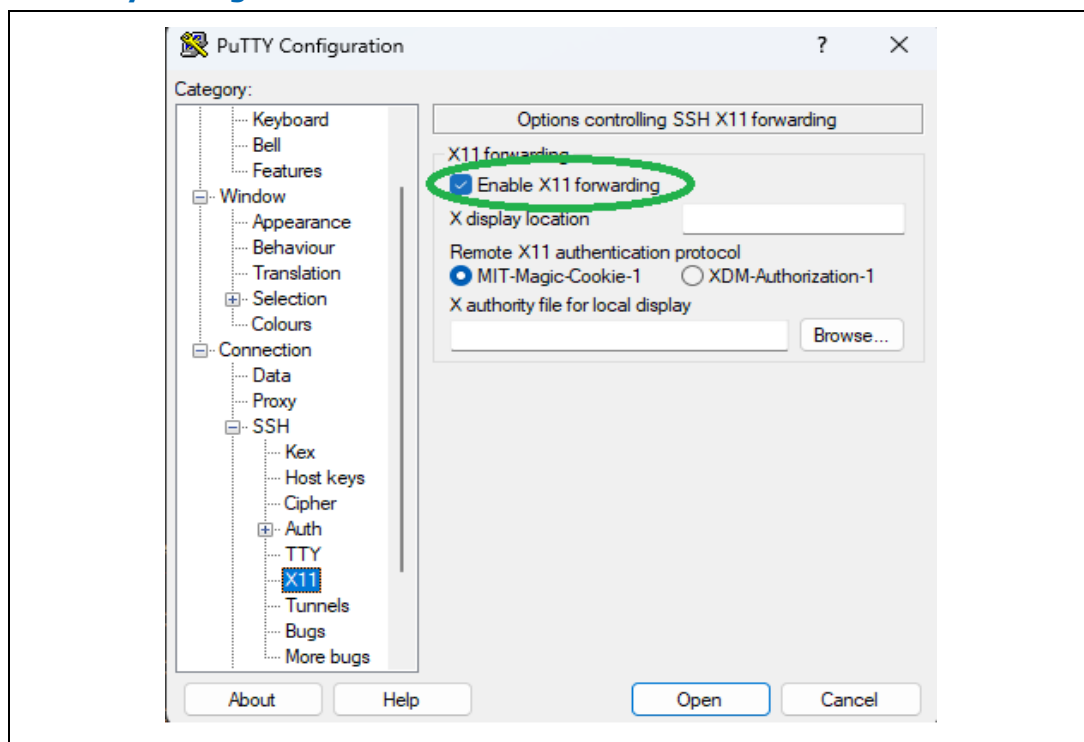
After login, verify that the DISPLAY environment variable is set by running 'echo \$DISPLAY'. You should see a value such as localhost:10.0. You can then run the VTS tool with the optional input parameter **-live_mon True** to open the live GPU monitoring window.

2.1.2 Using PuTTY on Windows*

If you are using PuTTY from a Windows client:

1. Open PuTTY and load your session.
2. Navigate to **Connection > SSH > X11**.
3. Enable 'Enable X11 forwarding'.
4. Ensure an X server (VcXsrv or Xming) is running on your Windows system before connecting.
5. Connect to the host system and run the VTS tool with the optional input parameter `-live_mon True`.

Figure 2-1. Putty Configuration

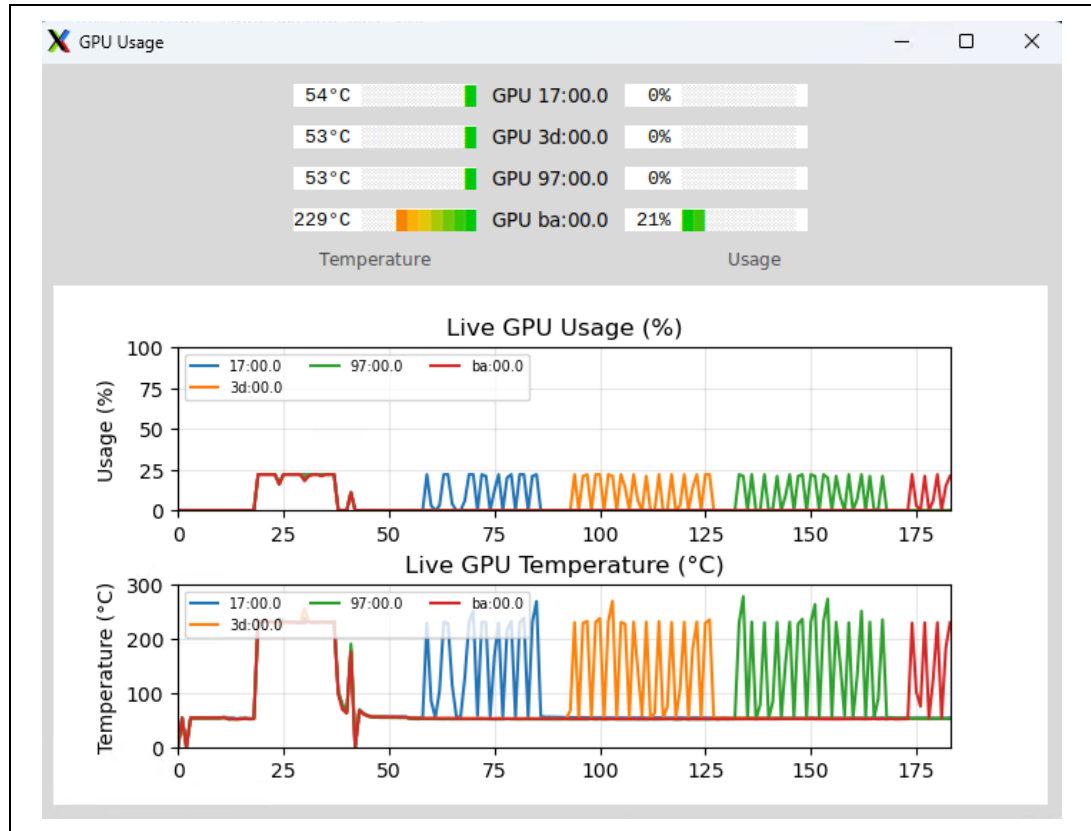


2.1.3 Enable Live Usage Monitoring

By default, GPU usage monitoring is disabled. To enable this feature, use the command line argument.

```
sudo python3 start_vts.py -live_mon True
```

Figure 2-2. GPU Usage Monitoring



3 Test Execution

The Intel® Data Center GPU Verification Test Suite (VTS) entry point is a Python* script called `start_vts.py` (also known as the test wrapper).

The VTS supports three different execution modes:

1. **Menu-Based Test Execution**
2. **Command Line Test Execution**
3. **JSON Configuration File Test Execution**

The general test execution flow is the same regardless of the execution mode used. The execution flow is as follows:

1. **GPU and CPU Discovery:** Available GPUs and CPUs in the System Under Test (SUT) are discovered and checked against the supported models.
2. **Requirement Verification:** Confirms all required Linux libraries, python packages, and tools are installed. Missing packages are automatically installed by the wrapper.
3. **Input parameter parsing:** Parses and validates all provided input parameters.
4. **GPU Details:** Prints a list of the available GPUs and their capabilities, SBDF, VID, DID, Speed, Width, and so on.
5. **System pre-flight checks:** Validate the OS, Python* version, required commands, GPU drivers, system memory, RAM speed, disk space, system load, leftover processes, and PCIe* alignment before test execution — failing on critical issues and prompting the user on warnings.
6. **Run Test:** Starts the selected test.
7. **Results Parsing:** Checks the results and provide a pass or fail.

3.1 Menu Test Execution

To start the VTS wrapper in menu mode, run the python script `start_vts.py` without any parameters as follows:

```
sudo python3 start_vts.py
```

In this execution mode, the test wrapper provides users with a simple menu interface to launch the different tests outlined in this guide. Most of the configurable inputs for the various tests have been predefined. There is also an option to run all available tests, run the debug script, and collect logs.

Once the Wrapper **Tests Menu** loads, select a number to start the corresponding test.

Figure 3-1. Tests Menu

```

*****
                                TESTS MENU
*****

1 : GPU Health Check
2 : GPU Environment and Device Check
3 : PCIe Bandwidth Test
4 : PCIe Lane Margin Test (LMT)
5 : Reset Test
6 : Memory Bandwidth Test
7 : Memory Stress Test
8 : Power and Thermal Stress Test
9 : Excursion Design Power Stress Test
10 : Functional Test (Not Available Yet)
11 : vLLM Test
12 : ML Perf Test
13 : OneCCL Collective Test (Not Available Yet)
14 : OneCCL Point to Point Test (Not Available Yet)
a : Run All
d : Debug Script
c : Collect Logs
q : Quit
*****

```

See Theory of Operations for specific details and pass or fail criteria for each test. Logs and run histories are saved in the `./logs/` folder relative to the wrapper menu script's path. Logfile names get appended with Pass or Fail. Test logs are saved as follows:

```

./logs/<Test_name>_execution_<year><month><day>_<time>
H:M:S>_test<Pass|Fail>.log

```

After each test is completed, except for the system-level reset tests, a summary with a Passed or Failed result and the wrapper menu will be displayed. In some cases, back scrolling may be required to see the results.

In this execution mode, most tests are preloaded with default values. Those requiring extra input parameters are explained next.

3.1.1 Reset Tests

The reset tests include host-level and device-level reset options. Host-level reset options include warm reset, cold DC cycle, soft reset, and a custom option.

Device-level reset options include Function-Level Reset (FLR), Secondary Bus Reset (SBR), link disable and enable cycling, link speed cycling, and link retrain.

To execute the **Reset Tests**, select option **5** from the Wrapper **Tests Menu**. The wrapper prompts the user to input the test type and iterations. For the custom reset option, a user-defined script can be executed by the reset test to issue chassis power commands, PDU commands, or to control other power infrastructure for AC cycle testing.

After completing warm, cold, soft, or custom reset tests, the wrapper will not restart on its own and must be launched by the user. Other reset test types return to the menu when completed.

For G3 cycle testing, use the *custom* option paired with a user written `.py` or `.sh` script for commanding chassis management controllers if virtual AC cycles are supported, or to control remotely operable power infrastructure, such as PDUs.

Some tests, such as Link Disable testing may cause the target devices to lose their Memory-Mapped I/O (MMIO) configuration and may require a host reboot when completed.

To terminate a custom, cold, soft, or warm reset test early, boot the OS to the recovery shell by adding a **1** to the kernel parameters in the boot menu during startup, then disable the reset test service using `systemctl disable rebootTest.service`. Then reboot or boot to `multi-user.target` using `systemctl isolate multi-user.target` and go to the `./reset/` test directory in the verification package and run `./systemReboot.sh clean`.

To terminate other test types early, press **Ctrl+C** during the health checks. Canceling the test during any other time may leave the target devices in a bad state, requiring a host reboot. The *clean* option will attempt to restore devices for some test types, but in some cases, a reboot may be required.

Reset Results

Results for **testonly**, **flr**, **sbr**, **linkchange**, **retrain**, and **linkdisable** will display *Test Passed* or *Test Failed* after each test completes, followed by the wrapper menu displaying. For reset tests involving host reboots, refer to the reset log files for results.

3.1.2 Power and Thermal Stress Test

The power and thermal stress test will prompt the user for the desired test time in seconds. The minimum value is 5 and the maximum value is 172800. The recommended value for this test is 900.

3.1.3 Excursion Design Power Stress Test

The excursion design power stress test will prompt the user for the desired test duration in seconds. The minimum value is 5 and the maximum value is 172800. The recommended value for this test is 900. The test stresses the target device for a set duration, then rests for a set duration, and repeats. By

default, the time under stress is three seconds and the time in idle is three seconds.

3.1.4 vLLM Benchmark Test

3.1.4.1 Prerequisites

Before running the test, ensure the following requirements are met:

- Docker* installed and running
If the Docker daemon is not running, you may see an error such as:

```
Docker daemon not running. Start it with: sudo systemctl start docker
```

- Docker Hub* credentials
 - Docker Hub username
 - Docker Hub *access token* (not your password)The test validates credential presence and performs docker login – password. If credentials are missing, execution stops with a clear error message.
- Intel GPU/XPU drivers installed and devices visible
Devices are discovered via the VTS device manager. The tensor-parallel degree is derived from the detected devices.
- Optional: Proxy configuration
If operating behind a corporate proxy, configure HTTP_PROXY, HTTPS_PROXY, and NO_PROXY in the .env file. The test can configure a Docker systemd drop-in and restart Docker to activate proxy settings.

Note: Never paste real Docker* or Hugging Face* access tokens into shared logs, emails, or screenshots. Treat tokens as passwords and store them securely.

Configuration File (.env)

The base configuration is loaded from:

```
gpu/bmg/tests/ai_wl/.env
```

Key Environment Variables

- Docker Authentication
 - DOCKER_USER: Docker Hub username
 - DOCKER_TOKEN: Docker Hub access token
- Server Configuration
 - MODEL: Hugging Face model identifier (for example: deepseek*-ai/DeepSeek-R1-Distill-Qwen-14B)
 - HOST: Server bind address (default: 0.0.0.0)
 - PORT: Server port (default: 8000)

- Performance and Behavior
 - DTYPE: Data type (for example: float16)
 - GPU_MEMORY_UTIL: Fraction of GPU memory to use (default: 0.75)
 - ENABLE_PREFIX_CACHING: Enable prefix caching (true / false)
 - DISABLE_LOG_REQUESTS: Disable per-request logging
 - MAX_MODEL_LEN: Maximum model context length
 - MAX_NUM_BATCHED_TOKENS: Maximum tokens per batch
 - BLOCK_SIZE: KV cache block size
 - TRUST_REMOTE_CODE: Allow execution of remote model code
 - TENSOR_PARALLEL_SIZE: Tensor parallel degree
 - DATA_PARALLEL_SIZE: Data parallel replicas
 - ENABLE_EXPERT_PARALLEL – Enable expert parallelism (MoE models only)
- Proxy
 - HTTP_PROXY
 - HTTPS_PROXY
 - NO_PROXY
- Benchmark Parameters
 - DATASET_NAME – Input source (random or sonnet)
 - INPUT_LEN – Input token length
 - OUTPUT_LEN – Output token length
 - NUM_PROMPT – Number of prompts
 - REQUEST_RATE – Request rate (default: inf)
 - BACKEND – Benchmark backend (default: vllm)

3.1.4.2 Run Test

To execute the **vLLM Test**, select option **11** from the Wrapper **Tests Menu**.

The wrapper prompts the user to input Model name, Tensor parallel size, Maximum Concurrency, input length and output length. vLLM service and benchmark test take the rest of the parameters from .env file specified in the [Prerequisites](#) section.

3.1.5 ML Perf Benchmark Test

3.1.5.1 Prerequisites

Before running the test, ensure the following requirements are met:

Configuration File

In `./VTS/gpu/bmg/tests/ai_wl/` folder, copy `config.env.template` to be e.g. `config.env`.

Proxy Server Setup

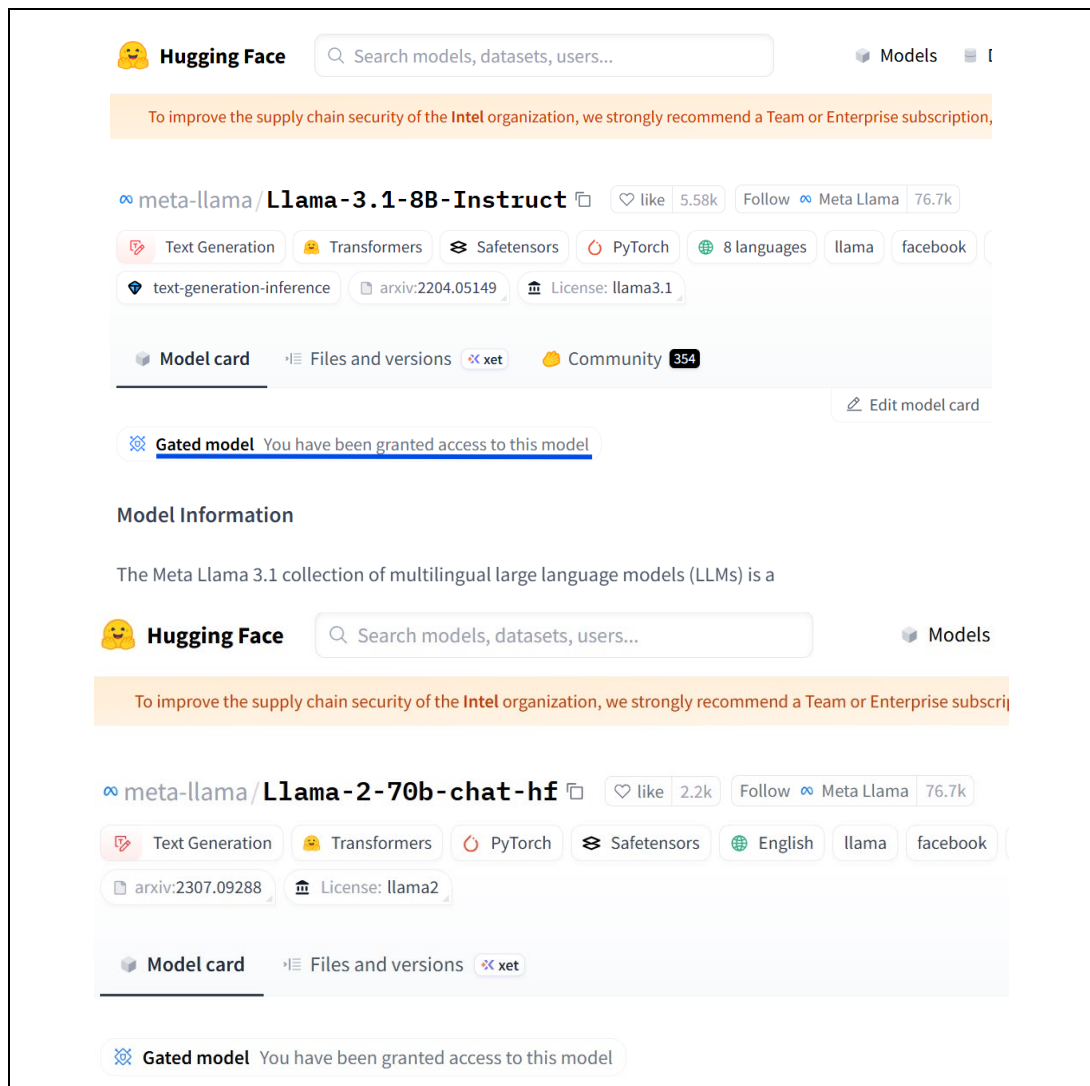
If your test environment is behind proxy server, make sure you fill up with your `http_proxy` and `https_proxy` server values in `config.env` file as follows.

```
# Proxy Configuration
# Set your corporate/network proxy here, or leave empty if not needed.
# These are passed into Docker containers and subprocess calls.
HTTP_PROXY=
HTTPS_PROXY=
```

Model Access to Hugging Face

1. Create an account for Hugging Face website.
2. Make sure you have been granted access to the models.

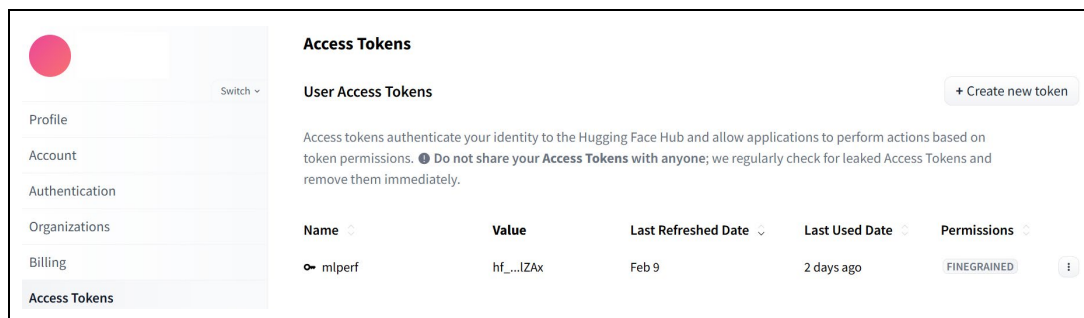
Figure 3-2. Access to the Models



<https://huggingface.co/meta-llama/Llama-3.1-8B-Instruct>
<https://huggingface.co/meta-llama/Llama-2-70b-chat-hf>

3. Create your own access token for model download.
Token type to be fine-grained (default).
<https://huggingface.co/settings/tokens>

Figure 3-3. Access Token for Model Download



Docker Daemon Proxy Configuration

1. Edit `/etc/systemd/system/docker.service.d/http-proxy.conf` file with values below inside the conf file; if not, create corresponding folder and file, and fill up with values as follows:

```
[Service]
Environment="HTTP_PROXY=<your_http_proxy>"
Environment="HTTPS_PROXY=<your_https_proxy>"
Environment="NO_PROXY=localhost,127.0.0.1"
```
2. When modification is done and you have to reload and restart the Docker daemon with commands as follows:

```
sudo systemctl daemon-reload
sudo systemctl restart docker
```

Download Model

For Llama*3.1-8b model hf format download:

```
export HF_TOKEN='hf_your_token_here'
sudo -E python3 start_vts.py -tn 12 --download_model 'llama3.1-8b' --
hf_token $hf_your_token_here
```

For Llama2-70b model hf format download:

```
export HF_TOKEN='hf_your_token_here'
sudo -E python3 start_vts.py -tn 12 --download_model 'llama2-70b' --
hf_token $hf_your_token_here
$hf_your_token_here: it's the Hugging Face fine-grained access token
created above.
```

3.1.5.2 Run Test

To execute the MLPerf Test, select **option 12** from the **Wrapper Tests Menu**.

The wrapper prompts the user to input docker image to run MLPerf test, data directory, model directory, model type, scenario, and mode. Verification test suite takes the rest of the parameters from config.env file specified in the section Prerequisites.

3.2 CMD Line Test Execution

Each test can have multiple configurable parameters. If a user wants to run a certain test with values other than the defaults offered through the test menu, the test can be called from the command line with the desired parameters. The following example shows the parameters that apply to every test, except *test_specific_arguments*, which are defined per test:

```
sudo python3 start_vts.py <-tn test_number> [-rep repetitions] [-mt
gpu_monitor_type>] [-cs cpu_stress_type] [-pcie_downgrade
pcie_downgrade_enable] [-d debug_logs] [-stop_on_error stop_on_error] [-
live_mon live_mon] {test_specific_arguments}
```

Where:

- *test_number*: Indicator of the test to execute.
 - 1: GPU Health Check
 - 2: GPU Environment and Device Check
 - 3: PCIe Bandwidth Test
 - 4: PCIe Lane Margin Test (LMT)
 - 5: Reset Test
 - 6: Memory Bandwidth Test
 - 7: Memory Stress Test
 - 8: Power and Thermal Stress Test
 - 9: Excursion Design Power Stress Test
 - 10: Functional Test (Not Available Yet)
 - 11: vLLM Test
 - 12: ML Perf Test
 - 13: OneCCL Collective Test (Not Available Yet)
 - 14: OneCCL Point to Point Test (Not Available Yet)
 - a: Run All
 - d: Debug Script
 - c: Collect Logs
- *repetitions*: Number of repetitions to run the test (default: 1).
- *gpu_monitor_type*: The GPU monitor type: xpum, dgdiag, and None (default: xpum). Intel® XPU Manager (Intel® XPUM) provides high-level operational GPU telemetry suitable for monitoring and deployment, while Intel® Discrete Graphics Diagnostic Tool (Intel® DG Diagnostic Tool) captures deep hardware-level diagnostic data required for validation, debugging, and understanding low-level power, thermal, and firmware behavior.
- *cpu_stress_type*: The CPU stress tool to run in parallel: None, stress-ng, and ptat (default: None). The CPU stress type stress-ng applies a load to the CPU, but only provides an aggregate measurement point for power,

frequency, and temperature (one core, one temperature sensor, and so on). The CPU stress type PTAT stresses the CPU and delivers a comprehensive architecture-aware set of telemetry that provides readings from multiple CPU sensors for temperature, power, and frequency. So, PTAT is significantly more effective for characterization and initial silicon testing, whereas stress-ng functions primarily as a simple workload generator with limited insight suitable for GPU focused validation.

- *pcie_downgrade_enable*: Enable or Disable PCIe Downgrade feature: True for disabling it and False for keeping it as is.
- *debug_logs*: Collect debug logs after each test execution: True and False (default: False).
- *stop_on_error*: Can be True or False. If True then execution will stop if test fails.
- *live_mon*: Can be True or False. If True then GPU Live Monitoring will be launched in parallel to test execution.
- *test_specific_arguments*: See the following sections for specific test arguments.

For a complete list of options for each test, call the wrapper help by running the following command:

```
sudo python3 start_vts.py -h
```

The options listed previously might have different default values depending on the test, for instance, the power and thermal stress test defaults to stress-ng for CPU stress, but the PCIe bandwidth test defaults to nothing for the CPU stress. See the following sections for specific test details.

3.2.1 GPU Health Check

To execute the test, use:

```
sudo python3 start_vts.py -tn 1
```

There are no *test_specific_arguments* for the GPU health check test.

3.2.2 GPU Environment and Device Check

To execute the test, use:

```
sudo python3 start_vts.py -tn 2 [-l diag_level]
```

Where:

- *diag_level*: Diagnostic level: quick, medium, and long (default: long)

3.2.3 PCIe Bandwidth Test

To execute the test, use:



```
sudo python3 start_vts.py -tn 3 [-inst instance] [-mode mode] [-dir
direction] [-engine engine] [-tool tool] [-iterations iterations] [-size
size]
```

Where:

- *instance*: GPU device instance, -1 for all devices (default: -1)
- *mode*: Execution mode: serial, parallel, all (default: all)
- *direction*: Traffic direction: h2d, d2h, bidirectional, all (default: all)
- *engine*: Traffic engine: copy, compute, all (default: all)
- *tool*: Bandwidth test tool: ze_bandwidth, memory_benchmark_I0, all (default: all)
- *iterations*: Number of test iterations (default: 500)
- *size*: Buffer size in bytes (default: 268435456)

If all is selected for mode, direction, engine and tool (which are the default for all of them) then a subset of all the possible combinations will be run, consisting of:

Table 3-1. PCIe Bandwidth Tests Subset

Mode	Tool	Direction	Engine
serial	ze_bandwidth	d2h	copy
serial	ze_bandwidth	d2h	compute
serial	ze_bandwidth	h2d	copy
serial	ze_bandwidth	h2d	compute
parallel	ze_bandwidth	d2h	copy
parallel	ze_bandwidth	d2h	compute
parallel	ze_bandwidth	h2d	copy
parallel	ze_bandwidth	h2d	compute

3.2.4 PCIe Lane Margin Test

To execute the test, use:

```
sudo python3 start_vts.py -tn 4 [-n iterations] [-rn receiver number]
```

Where:

- *iterations*: Number of iterations (default: 1)
- *receiver number*: Receiver Number to margin (default: 6 for GPU Rx)

3.2.5 Reset Test

To execute the test, use:

```
sudo python3 start_vts.py -tn 5 [-rt reset_type] [-iterations iterations]
[-custom-script custom_script]
```

Where:

- *reset_type*: Can be flr, sbr, warm, cold, soft, linkdisable, linkchange, retrain, custom, testonly, clean (default: testonly)
- *iterations*: Number of iterations (default: 1)
- *custom_script*: Custom script path for custom reset type

3.2.6 Memory Bandwidth Test

To execute the test, use:

```
sudo python3 start_vts.py -tn 6 [-inst instance]
```

Where:

- *instance*: GPU device instance, -1 for all devices (default: -1)

3.2.7 Memory Stress Test

To execute the test, use:

```
sudo python3 start_vts.py -tn 7 [-inst instance] [-testtime test_time] [-stime sampling_time]
```

Where:

- *instance*: GPU device instance, -1 for all devices (default: -1)
- *test_time*: Total monitoring time in seconds (default: 60).
- *sampling_time*: Sampling time interval in msec, 0 is to disable Sampling (default: 0).

3.2.8 Power and Thermal Stress Test

To execute the test, use:

```
sudo python3 start_vts.py -tn 8 [-inst instance] [-testtime test_time]
```

Where:

- *instance*: GPU device instance, -1 for all devices (default: -1)
- *test_time*: Total test time in seconds (default: 300).

Notes:

- For this test, the default for *-cs* is *stress-ng*.
- For this test, the only supported value for *-mt* is *xpum*.

3.2.9 Excursion Desing Power Stress Test

To execute the test, use:

```
sudo python3 start_vts.py -tn 9 [-inst instance] [-testtime test_time] [-at active_time] [-it idle_time]
```

Where:

- *instance*: GPU device instance, -1 for all devices (default: -1)
- *test_time*: Total test time in seconds (default: 300).
- *active_time*: Active time for stress cycles in seconds (default: 3).
- *idle_time*: Idle time between stress cycles in seconds (default: 3).

Notes:

- For this test, the default for *-cs* is *stress-ng*.
- For this test, the only supported value for *-mt* is *xpum*.

3.2.10 Functional Test

To execute the test, use:

```
sudo python3 start_vts.py -tn 10
```

There are no *test_specific_arguments* for the functional test as they are not available yet.

3.2.11 vLLM Test

Before executing the test, verify that all prerequisites have been updated as specified in the section [3.1.4.1](#).

To execute the test, use:

```
sudo python3 start_vts.py -tn 11 [--model MODEL] [--enforce-eager] [--trust-remote-code] [--host HOST] [--port PORT] [--disable-log-requests] [--gpu-memory-util FLOAT] [--block-size INT] [--max-num-batched-tokens INT] [--max-model-len INT] [--input-len INT] [--output-len INT] [--disable-sliding-window] [--max-concurrency INT] [--tp INT] [--data-parallel-size INT] [--enable-expert-parallel] [--dtype DTYPE] [--quantization TYPE] [--enable-prefix-caching] [--dataset-type TYPE] [--docker-user USER] [--docker-token TOKEN] [--docker-image IMAGE] [--http-proxy URL] [--https-proxy URL] [--no-proxy HOSTS] [--hf-token TOKEN]
```

The following is a list of command-line arguments available for configuring and running the vLLM service. These arguments are optional; if not provided by the user, values will be sourced from the *.env* file.

- **--model**: Specifies the model name or path to be served. It can be a local path or Hugging Face model ID.

- **--enforce-eager:** Forces eager execution mode. Useful for debugging but may reduce performance.
- **--trust-remote-code:** Allows execution of custom model code from remote repositories.
- **--host:** Host address to bind the vLLM server.
- **--port:** Port number on which the vLLM service listens.
- **--disable-log-requests:** Disables request-level logging for improved performance.
- **--gpu-memory-util:** Fraction of total GPU memory allowed for vLLM usage.
- **--block-size:** KV cache block size in tokens.
- **--max-num-batched-tokens:** Maximum number of tokens processed in a single batch.
- **--max-model-len:** Maximum context length supported by the model.
- **--input-len:** Expected input token length for benchmarking.
- **--output-len:** Expected output token length for benchmarking.
- **--disable-sliding-window:** Disables sliding-window attention mechanism.
- **--max-concurrency:** Maximum number of concurrent requests.
- **--tp:** Tensor parallel size.
- **--data-parallel-size:** Number of data-parallel replicas.
- **--enable-expert-parallel:** Enables expert parallelism for MoE models.
- **--dtype:** Computation data type (float32, float16, bfloat16).
- **--quantization:** Quantization type used to reduce memory footprint.
- **--enable-prefix-caching:** Enables KV cache reuse for repeated prompt prefixes.
- **--dataset-type:** Specifies dataset format for benchmarking or offline inference.
- **--docker-user:** Docker registry username.
- **--docker-token:** Docker registry access token.
- **--docker-image:** Docker image name used to launch vLLM.
- **--http-proxy:** HTTP proxy server address.
- **--https-proxy:** HTTPS proxy server address.
- **--no-proxy:** Comma-separated list of hosts to bypass proxy.
- **--hf-token:** Hugging Face access token for private or gated models.

3.2.12 ML Perf Test

To execute the test, use:

```
sudo -E python3 start_vts.py -tn 12 [--data_dir PATH] [--model_dir PATH]
[--model_type TYPE] [--scenario SCENARIO] [--mode MODE] [--download_model
MODEL] [--hf_token TOKEN] [--docker_image IMAGE]
```

The following is a list of command-line arguments available for configuring and running the ML Perf Test. These arguments are optional; if not provided by the user, values will be sourced from the `config.env` file.

- **--data_dir:** Path to dataset directory on the host, mounted as '/data' inside the container
- **--model_dir:** Path to model directory on the host, mounted as '/model' inside the container
- **--model_type:** Model type to benchmark (llama3_1-8b, llama2-70b)
- **--scenario:** MLPerf benchmark scenario (Offline, Server)
- **--mode:** Benchmark mode (Performance, Accuracy, Compliance)
- **--download_model:** Download a Hugging Face model before running the test. Accepts model aliases (for example, 'llama3.1-8b' resolves to 'meta-llama/Llama-3.1-8B-Instruct') or a full Hugging Face repo ID. Leave empty to skip.
- **--hf_token:** Hugging Face access token, required for downloading gated models. It can also be set via the 'HF_TOKEN' environment variable.
- **--docker_image:** Docker image to use for MLPerf testing

Example

Run Offline Performance benchmark with Llama 3.1-8B (model and dataset already downloaded):

```
sudo -E python3 start_vts.py -tn 12 --scenario Offline --mode Performance --
model_type llama3_1-8b
```

Note: Llama2-70b support is under development.

3.2.13 OneCCL Collective Test

To execute the test, use:

```
sudo python3 start_vts.py -tn 13
```

There are no *test_specific_arguments* for the OneCCL collective test as they are not available yet.

3.2.14 OneCCL Point to Point Test

To execute the test, use:

```
sudo python3 start_vts.py -tn 14
```

There are no *test_specific_arguments* for the OneCCL point to point test as they are not available yet.

3.2.15 Run All

To execute the test, use:

```
sudo python3 start_vts.py -tn a
```

There are no *test_specific_arguments* for the run all option, the list of tests to be executed is defined at: `./gpu/<product>/tests/run_all.json`

Where:

- *product* is the Intel® Data Center GPU acronym for instance *bmg* for Battlemage.

3.2.16 Debug Script

To execute the test, use:

```
sudo python3 start_vts.py -tn d
```

There are no *test_specific_arguments* for the Debug Script. The resultant logs can be found in a timestamped .zip at:

```
./logs/debugLogs-<YYYY>-<MM>-<DD>_<hh><mm><ss>.zip
```

3.2.17 Collect Logs

To collect available test execution logs and create an HTML summary report, use:

```
sudo python3 start_vts.py -tn c
```

There are no *test_specific_arguments* for the collect logs option. The resultant logs can be found in a timestamped .zip at:

```
./VerificationTestSuiteLogs_<YYYY><MM><DD>_<hh><mm><ss>.zip
```

3.3 JSON Config File Execution

For automation purposes, a JSON file can be written to define a set of tests and parameters for the test wrapper to execute.

To execute a set of tests from a JSON configuration file, use:

```
sudo python3 start_vts.py -tc path_to_json_file
```

Where:



- *path_to_json_file*: Absolute path to a JSON file defining a set of tests to be executed. Example JSON files can be found at:
`./gpu/<product>/tests/`

Where:

- *product* is the Intel® Data Center GPU acronym for instance *bmg* for Battlemage.

4 *Theory of Operations*

This section describes how each test works in detail, including tests outside the wrapper, available test parameters, and the pass or fail criteria for each test.

On execution, the tests described in this section gather static information about the GPUs like PCIe* bus information (current and maximum links speed and width, SBDF, SVID, and SDID).

If a bash script in the test suite is not executable, set the execute bit using:

```
sudo chmod +x ./tests/<test_script>
```

Execute the VTS test wrapper script (`start_vts.py`). Each test script will check for prerequisites.

4.1 GPU Health Check

4.1.1 Low Level Test Description

The GPU Health Check validates the operational health of all Intel® Data Center GPUs in the system. It verifies the core temperature, memory temperature, power consumption, and frequency stability for each GPU and is within expected ranges and reports an "OK" status for each metric.

4.1.2 Pass or Fail Criteria

To be updated in a future release.

4.2 GPU Environment and Device Check

4.2.1 Low Level Test Description

The GPU Environment and Device Check verifies the Intel® Data Center GPU hardware and software environments are configured correctly. It detects all Intel® Data Center GPU devices within the system and verifies all driver versions are correct and matching, and all necessary accelerator compute libraries are installed and accessible.

4.2.2 Pass or Fail Criteria

To be updated in a future release.

4.3 PCIe Bandwidth Test

4.3.1 Low Level Test Description

The PCIe* bandwidth test measures PCIe* bandwidth while moving data from Host to Device (H2D), Device to Host (D2H) and bidirectionally. It supports a serial mode where it operates on one card at a time, or a parallel mode where it operates on all cards simultaneously. It also supports using copy or compute engines to move the data. The tools underneath the PCIe bandwidth test are `ze_bandwidth` and `memory_benchmark_I0`.

4.3.2 Pass or Fail Criteria

The calculation of the PCIe bandwidth pass/fail threshold is performed through a series of methodical steps to ensure the criteria accurately reflect realistic hardware performance.

The process begins by determining the theoretical maximum bandwidth, which will depend on the PCIe speed and width.

$$\text{Theoretical Maximum Bandwidth (GB/s)} = \text{Speed (GT/s)} \times \text{Number of Lanes} \times \text{Encoding Efficiency} \div 8$$

Where:

Speed (GT/s): This is the PCIe transfer rate.

Number of Lanes: How many PCIe lanes are used.

Encoding Efficiency: Varies based on PCIe version:

For PCIe 1.0 and 2.0 (up to 5.0 GT/s), efficiency is **80%** due to 8b/10b encoding.

For PCIe 3.0 and newer (above 5.0 GT/s), efficiency increases to **98.46%** because of 128b/130b encoding.

(÷ 8): This converts bits to bytes.

Once the theoretical bandwidth is established, it is refined using several platform-specific correction factors. These adjustments account for performance limitations encountered in real-world environments:

- **Achievable Factor:** Defaulted to 95% of the theoretical maximum, representing typical real-world attainability.
- **Part-to-Part Variation Factor:** Set at 95%, this factor addresses manufacturing differences between individual hardware components.
- **Temperature Impact Factor:** At 99%, this corrects for performance reductions due to thermal conditions.
- **Standard Deviation Impact Factor:** Also set at 99%, this accounts for statistical variations in test results.

These factors are multiplied together with the theoretical bandwidth value to establish a realistic baseline bandwidth.

Adjusted Bandwidth = Theoretical Bandwidth × Achievable Factor × Part-to-Part Variation Factor × Temperature Impact Factor × Standard Deviation Impact Factor

For bidirectional tests, the adjusted bandwidth baseline is doubled to account for simultaneous read and write operations occurring across the PCIe interface.

Further refinements are made by applying engine-specific overhead factors. These overheads vary depending on the test engine type (compute versus copy) and the direction of data transfer (host-to-device, device-to-host, or bidirectional). Different overhead ratios are calculated for each scenario, using formulas like $(256/(256+20))$ to quantify protocol and engine-related inefficiencies.

The final pass/fail threshold is the product of all these adjustments. The result is rounded to two decimal places and represents the minimum acceptable bandwidth for the test. This approach ensures the criteria account for hardware limitations, thermal effects, part-to-part variations, and protocol overhead, establishing a standard based on achievable performance rather than on theoretical maximums.

4.4 PCIe Lane Margin Test

4.4.1 Low Level Test Description

The PCIe lane margin test measures the “Eye Margin” by sweeping the phase and voltage parameters to measure the PCIe voltage margin and timing. This allows users to analyze signal quality and find issues that can cause link failures and helps identify weak lanes by providing per lane results.

If the eye margin test starts from the Wrapper **Tests Menu** by selecting option **4**, the margin test automatically runs on all PCIe lanes on all available GPUs in the system.

Alternatively, the user can choose to run the test from the command line and configure specific test input parameters.

Note: PCIe Lane Margin script supports Python 3.10, 3.12, and 3.13 respectively.

4.4.2 Pass or Fail Criteria

The PCIe Lane Margin Test passes if the design meets the minimum eye width and height for reliable GPU data reception; otherwise, it fails. Results are measured per GPU and per lane.

4.5 Reset Tests

The reset cycle script can perform various reset types, including host hard S5 power cycle, host soft S5 power cycle, warm host reset, device Secondary Bus Reset (SBR), device Function-Level Reset (FLR), device link disable and enable cycling, link speed change cycling, device link retrain cycling, or call a custom script to control a PDU or run OOB commands to execute other reset types such as G3 power cycle.

The script requires root or sudo privileges. For AC (G3) cycle testing, a PDU with network controllable outlets or other controllable power infrastructure is required. For AC cycles, set the host BIOS to resume after AC loss. If script requirements are not met for a test, the script will exit with an error describing the problem.

All non-host level resets require the system to be in the multi-user runlevel target, or the script will exit. This is to avoid conflicts with graphic environment overhead on device drivers.

4.5.1 Setup

If the script is not executable, set the execute bit using:

```
sudo chmod +x systemReboot.sh
```

The **Reset Tests** require the Intel® Arc™ Pro graphics drivers, xpu-smi from the intel-ppa repository, and ipmitool and libpciaccess-dev from the default OS package repos.

4.5.2 Execution Flow

At launch, the reset cycle test script collects system info and generates a temporary configuration file based on the selected test type. For cold, soft, warm, and custom reset types, the script also creates a systemd unit file to automatically re-launch the script after each reset event. The script also temporarily copies itself to a SELinux compatible directory path for systemd service execution.

After each reset, the script checks the targeted devices under test to confirm device functionality, enumeration, link speed, and link width, and compares the devices' status to a hardware inventory initially generated by the script.

4.5.3 Use

4.5.3.1 Terminating a custom, cold, soft, or warm Reset Test Early

Tests involving dozens or hundreds of system resets can take many hours. If a test needs to be terminated before the defined repetitions run out, follow these steps to disable the reset service.

1. Boot the OS to runlevel 1.

2. Use the recovery shell to disable the reset test service using `systemctl disable rebootTest.service`.
3. Boot to multi-user.target using `systemctl isolate runlevel3.target`.
4. Go to the `./Reset/` directory in the verification package and run `./systemReboot.sh clean`.

4.5.3.2 Terminating flr, sbr, linkdisable, linkchange, or retrain Tests Early:

1. Monitor the test on the console and wait for the health check portion to begin.
2. Press **Ctrl+C** during the health checks.
3. Run `./systemReboot.sh clean`.

Note: Canceling these tests at any time other than during the health checks can leave the target devices in a bad state and will require a reboot to restore device functionality.

4.5.3.3 Starting the Reset Script from the Command Line

The script requires multiple command line parameters: the number of cycles to run, the reset type, and if specifying *custom*, an optional bash or Python* script to call for controlling special hardware.

Example command lines:

```
./systemReboot.sh <num_cycles> <reset_type>
./systemReboot.sh <num_cycles> custom <optional_script>
./systemReboot.sh <other_option>
```

Where:

- **<num_cycles>**: Number of cycles the script will execute a test
- **<reset_type>**: warm, cold, flr, sbr, custom, linkdisable, linkchange, retrain
 - **warm**: Runs a warm reset cycle test using the host OS reset function.
 - **cold**: Runs a host S5 power cycle using the host motherboard's management controller. Faster and more reliable than using soft resets.
 - **soft**: Runs a host S5 power cycle using the host OS to gracefully shut down the system and uses the system rtcwake timer to turn the power back on after a pre-defined duration. To change the power-on delay, edit the `systemReboot.sh` and modify the `_powerOnDelay` variable. Easier on the OS than using cold resets but takes significantly longer.
 - **flr**: Unbinds the driver for the devices under test and resets the endpoint, then rebinds the devices. After each cycle, device health is checked before executing a subsequent run.

- **sbr**: Stops the driver for the devices under test, removes the bus node downstream from the host, and issues a bus reset on the upstream port. After each cycle, the test rescans the PCI bus for devices, restarts the driver, and checks the device health before starting a subsequent run.
- **linkchange**: Cycles the target devices' link speed one step at a time, starting from the maximum supported speed down to 5 GT/s, and back up to the maximum supported speed.
- **retrain**: Initiates PCIe* link retrain on all Intel® Arc™ Pro GPUs in parallel and checks for PCIe* errors or downgrading.
- **linkdisable**: Stops the device driver and disables the PCIe* links to the target devices; then, re-enables the link, scans for devices, and checks the link training. This test may cause the target devices to stop responding, requiring a host reboot after the test completes.
- **custom**: By default, idly waits for a system reset event. After a reset event occurs, the script records the event and runs device health checks. If specified, after each reboot, this option can launch a user-provided script to control a PDU or other remote reset hardware.

Example command line:

```
./systemReboot.sh 10 custom optional_script.sh
```

- **<optional_script>**: A bash .sh or Python* .py script placed in the same directory the reset script resides in. The following example shell script can be called by the reset script to directly control a PDU. Using a while loop helps account for network issues or missed commands between the test system and the PDU by allowing the ssh command to retry until successful. The example script is based on a Raritan* Dominion* switchable PDU. Other PDUs may be used but require specific script implementation.

```
#!/bin/bash
while true; do
echo "power outlets #,# cycle /y" | sshpass -p <PDU_password>
ssh -T <PDU_username>@<PDU_IP_Address>
sleep 10
done
```

To use the preceding script, install sshpass and ssh into the PDU from the test system at least once to add the PDU ssh fingerprints to the test system's known hosts file before starting the test, or fingerprint prompts will block script execution. For the correct PDU commands, refer to the documentation for the specific PDU model used for testing.

- **<other_options>**: *clean*, *testonly*
 - **clean**: If a test is terminated early, the *clean* option erases the test configuration files and generates a log file for the terminated test.
 - **testonly**: This option runs the target device health checks without issuing any resets. After the check, the script generates a log file with the results.

4.5.4 Results and Pass/Fail Criteria

Each reset cycle test type has a recommended cycle count. The following recommended cycle counts can be adjusted up or down based on testing needs, system sample size, and available test time. The actual test times for most reset types are heavily dependent on the host system and number of target devices, so cycle times will not be provided.

warm, cold or soft, linkdisable = 500 cycles

AC cycle (using custom option) = 300 cycles

flr, sbr = 1500 cycles

linkchange, retrain = 1000 cycles

After each iteration, the reset script checks the hardware health for each device under test. This check includes a comparison between the PCIe* link speed and width, Base Address Register (BAR) assignment, and other device health checks.

If a device fails enumeration or deviates from its original healthy state during the reset test, the test will terminate, issue a failure message, and generate a log file containing the results and a basic failure reason.

If the test completes all iterations, a results log file is generated with the reset type, timestamp, and pass or fail in the filename. The log file will contain a test journal and end with either a "Test Passed" or "Test Failed" message and the failure reason.

4.6 Memory Bandwidth Test

4.6.1 Low Level Test Description

The memory bandwidth test measures the GPU memory subsystem bandwidth performance across different memory access patterns, validates memory controller efficiency, and ensures the memory subsystem meets specification.

Running it from the Wrapper **Tests Menu** by selecting option **6** will automatically run the test in all the available GPUs.

Alternatively, the user can choose to run the test from command line and be able to configure specific test input parameters.

4.6.2 Pass or Fail Criteria

The pass or fail criteria for the memory bandwidth test is as follows:

- Pass:
 - All devices report "Result: Pass" for the memory bandwidth diagnostic.

- No errors, timeouts, or warnings indicating degraded performance are present.
- Fail:
 - A device reports "Result: Fail."
 - A device encounters an execution error.
 - A device does not complete the memory bandwidth test.

4.7 Memory Stress Test

4.7.1 Low Level Test Description

A comprehensive GPU memory subsystem stress test that uses multiple stress testing methodologies. The test validates memory stability, data integrity, and error handling under sustained high-load conditions. It also identifies memory defects, thermal issues, and stability problems.

The test consists of two different stress phases run serially one after the other. On phase one, OpenCL kernel-based memory access patterns are used as the stressor. Phase two stresses memory by performing sustained high bandwidth memory operations.

If a user starts the test using option **7** from the Wrapper **Tests Menu**, it will automatically run on all available GPUs.

Alternatively, the user can choose to run the test from the command line to specify test input parameters.

4.7.2 Pass or Fail Criteria

There are three different tests embedded in the memory stress test. The pass/fail criteria for each of those tests is as follows:

1. **OpenCLMemTest:** The OpenCLMemTest is designed to rigorously evaluate DGfx memory by executing a series of test patterns using the OpenCL framework. The specific patterns implemented in this test include:
 - MovingInversionsOnesZeros
 - MovingInversionsRandom
 - verify_walkingtest
 - walkingZeros

These patterns are applied to the DGfx memory to ensure thorough coverage and stress of the memory subsystem. The test is considered a failure if any algorithm encounters issues such as memory allocation problems, read/write failures, or compare mismatches during execution.
2. **StressTest:** This test is used to stress the DGfx memory by writing inverted patterns back and forth to stress the memory. The test will fail if there are any failures in the algorithm, typical failures are: memory allocation issues, memory read write or compare failures.

3. **Memory Error:** Pass if no memory errors are detected or fail otherwise.

The overall test result will report a pass only the three preceding mentioned tests pass or report fail otherwise.

4.8 Power and Thermal Stress Test

4.8.1 Low Level Test Description

The power and thermal stress test uses a GFXStress Vulkan workload from Intel® DG Diagnostic Tool to validate power delivery, thermal management, and system stability under a continuous GPU load, while also stressing the CPU using the stress-ng Linux app or PTAT. The default and preferred application for CPU stress is stress-ng.

Select option **8** on the Wrapper **Tests Menu** and specify a test duration to run on all GPUs or use the command line to configure other input parameters.

4.8.2 Pass or Fail Criteria

The test has two metrics used to determine a pass or fail: measured GPU card power and thermal throttling detection, as defined in the following:

Power Criterion:

- PASS: Measured average power is equal to or above 95% of the configured Psys Limit 2.
- FAIL: Measured average power is less than 95% of the configured Psys Limit 2.

Thermal Criterion:

- PASS: No thermal throttling occurred during the test.
- FAIL: thermal throttling occurred during the test.

4.9 Excursion Design Power Stress Test

4.9.1 Low Level Test Description

The Excursion Design Power (EDP) stress test uses Intel® DG Diagnostic Tool PulseStress with adaptive pulse detection to verify the functionality of the power supply by generating a fast power usage transient from low power to high power and vice versa. The power cycles repeat throughout the test execution time.

Run the test on all GPUs by selecting option **9** on the Wrapper **Tests Menu** or use the command line to set specific input parameters and duration.

4.9.2 Pass or Fail Criteria

The test uses two metrics for the pass/fail criteria: GPU card power measurements and GPU thermal throttling conditions. The criteria are as follows:

Power Criterion:

- **PASS:** The test duration is based on the user specified time to run. This time is divided by how much time a full pulse cycle takes to determine how many cycles the test will run. For the test to pass, at least 90% of the expected number of power pulse cycles must be completed before the test time runs out, and the measured peak power must be greater than or equal to 95% of PsysL2.
- **FAIL:** The PASS conditions are not met.

Thermal Criterion:

- **PASS:** No thermal throttling occurs during execution.
- **FAIL:** Any instance of thermal throttling is observed.

4.10 Functional Test

Test planned for future release.

4.11 vLLM Test

4.11.1 Low Level Test Description

This section describes the end-to-end procedure for deploying, launching, and benchmarking the **vLLM inference server** on **Battlemage** GPUs running Ubuntu* Linux*. The workflow follows Intel-published driver installation guidance and uses the official intel/vllm:xpu Docker image.

Perform the following high-level steps:

1. Validate the environment of Docker engine availability by running the docker info command, and optional proxy configuration.
2. Authenticate to Docker Hub using the docker login command.
3. Pull the required vLLM Docker image (if not already present).
4. Start a Docker container named vllm_service_xpu with host networking and XPU device mappings.
5. Launch the vLLM OpenAI-compatible API server inside the container using parameters from the .env file and interactive menu overrides.
6. Wait for server readiness.
7. Execute the built-in vLLM CLI benchmark against the running server.

8. Collect and parse performance metrics.
9. Stop and remove the container unless reuse is explicitly enabled.

4.11.2 Configuration and Sanity Checks

1. Load environment values from `.env` (`gpu/bmg/ai_wl/.env`); if you provide command line arguments will override the corresponding environment value.
For example, if `.env` has `'model=meta-llama/Llama-2-7b'` and you pass `'-model meta-llama/Llama-3.1-8B-Instruct'`, the value is updated to `'meta-llama/Llama-3.1-8B-Instruct'`.
2. Perform a **model memory feasibility check** by parsing parameter count from the model name (for example, `meta-llama/Llama-2-7b`, the 7b is parameter count).
3. Estimate required GPU memory and compare it against available GPU memory.
4. Exit early if insufficient GPU memory is detected.

4.11.3 Docker Preparation

1. Log on to Docker Hub using provided credentials.
2. Pull the required vLLM image if it is not already available locally.

4.11.4 Container Lifecycle

1. Resolve port conflicts on the configured port (default: 8000).
2. Start the container with host networking, IPC sharing, cache volumes, and environment variables.
3. Display container status using `docker ps` and `docker stats`.

4.11.5 Server Startup and Readiness

The server is launched using:

```

1  python -m vllm.entrypoints.openai.api_server \
2  --model <MODEL> \
3  --dtype <DTYPE> \
4  --host <HOST> \
5  --port <PORT> \
6  --gpu-memory-util <VALUE> \
7  --max-num-batched-tokens <VALUE> \

```

```

8      --max-model-len <VALUE> \
9      --block-size <VALUE> \
10     --tp <TENSOR_PARALLEL_SIZE>
11

```

Logs are streamed and written to `/tmp/vllm-server.log`. Readiness is detected via log patterns (for example, "*Application startup complete*") and by polling the `/v1/models` endpoint until HTTP 200 is returned.

4.11.6 Benchmark Execution

The benchmark runs inside the container:

```

1      python -m vllm.entrypoints.cli.main bench serve \
2      --model <MODEL> \
3      --dataset-name <DATASET_NAME> \
4      --random-input-len <VALUE> \
5      --random-output-len <VALUE> \
6      --num-prompt <VALUE> \
7      --request-rate <VALUE> \
8      --backend vllm
9

```

Metrics are parsed and tagged in logs.

4.11.7 Cleanup

1. Stop and remove the container unless reuse is enabled.
2. Log total execution time.
3. Mark the test result as **Completed or Fail**.

4.11.8 Interpreting Results

Example benchmark output:

- Requests: 56
- Duration: 12.34 s
- Throughput: 123.45 tokens/s
- Latency (p50): 210 ms

- TTFT: 150 ms
- TPOT: 7.2 ms/token
- ITL: 1.8 ms/token

These values are aggregated into a metrics dictionary and logged at **BENCHMARK COMPLETE**.

4.12 MLPerf*

4.12.1 Low Level Test Description

MLPerf Inference is the industry-standard benchmark suite designed by the [MLCommons](#) consortium to measure how quickly machine learning (ML) systems can run trained models to make predictions from new data. It provides a standardized framework to ensure fair, "apples-to-apples" comparisons of hardware, software, and cloud platforms across different vendors.

Key Components and Design

- **The Load Generator (LoadGen):** A dedicated tool that mimics real-world traffic patterns by sending queries to the "System Under Test (SUT)".
- **Benchmarking Divisions**
- **Closed Division:** Strictly controlled to compare hardware and software optimizations using identical models and data.
- **Open Division:** Allows for architectural and algorithmic flexibility, serving as a proving ground for novel designs.
- **Diverse Model Coverage:** Benchmarks cover various AI tasks including image classification (for example, ResNet50), object detection (for example, SSD), speech-to-text (RNN-T), and increasingly, Large Language Models (LLMs) like Llama* 2/3.

Standardized Testing Scenarios

MLPerf Inference evaluates systems based on how they receive and process queries in the following scenarios:

- **Offline:** Processes an entire dataset as a single batch without latency constraints. **Metric:** Samples per second.
- **Server:** Simulates a web service where queries arrive at random intervals (Poisson distribution). **Metric:** Queries Per Second (QPS) within a strict latency bound.
- **Single-Stream:** Processes one sample at a time, typical of mobile devices. **Metric:** 90th percentile latency.

Here the focus is on inference benchmarks of llama2-70b and llama3.1-8b.

4.12.2 Pass or Fail Criteria

To pass MLPerf Inference benchmarking for **Llama 2-70B** (introduced in v4.0) and **Llama 3.1-8B** (introduced in v5.1), systems must meet strict **accuracy** and **latency** thresholds. These criteria ensure that performance optimizations (like quantization) do not degrade the model's functional quality beyond acceptable industry limits.

1. Accuracy Thresholds

Submissions in the "Closed Division" must achieve a specific percentage of the reference model's quality. Accuracy is measured using **ROUGE** scores (specifically ROUGE-1, ROUGE-2, and ROUGE-L) on a subset of the **CNN-DailyMail** dataset.

- **Standard Accuracy (99%):** Most submissions must reach at least **99% of the reference model's accuracy**.
- **High Accuracy (99.9%):** Certain categories (often labeled "-99.9") require the system to maintain near-perfect parity with the reference FP32/BF16 model.
- **Validation:** If a system fails to meet these ROUGE targets, the result is considered invalid and will not be published in the official leaderboards.

2. Latency Constraints (Server Scenario)

In the "Server Scenario," which simulates real-world random query arrivals, a system "passes" only if it processes at least 99% of queries within these specific latency bounds:

Metric	Llama 2-70B (v4.0+)	Llama 3.1-8B (v5.1+)
Time to First Token (TTFT)	≤ 2.0 seconds	≤ 2.0 seconds
Time Per Output Token (TPOT)	≤ 200 milliseconds	≤ 100 milliseconds

- **Interactive Scenario (Llama 3.1-8B only):** For highly responsive tasks like coding assistants, Llama 3.1-8B introduced a stricter "Interactive" tier with targets of **TTFT ≤ 0.5s** and **TPOT ≤ 30ms**.

3. Performance Metrics

While not "pass/fail" in the binary sense, a system's final score is only recorded if the above criteria are met. The primary performance metrics reported are:

Throughput: Tokens per second (Offline scenario) or Queries per second (Server scenario).

4.13 OneCCL Collective Test

Test planned for future release.

4.14 One CCL Point to Point Test

Test planned for future release.

4.15 Debug Script

The debug script collects various logs, checks configuration details, and device status information to help debug or troubleshoot configuration or hardware issues. The script does not accept any input parameters.

The debug script can be used as a basic configuration check for a new system setup, or for collecting system information after workload or device failures are seen.

After execution, the script generates a quick pass or fail summary for each device and test, as shown in the following sample image:

Figure 4-1. Example InfoCollect Result Summary Output

```
*****START_DEVICE_SUMMARY*****
Device status summary:
B60_Devices_Found_2
Device_0000:b0:00.0_BAR_Check_PASSED
Device_0000:c0:00.0_BAR_Check_PASSED
Device_0000:ae:00.0_PCI_Link_Check_PASSED
Device_0000:be:00.0_PCI_Link_Check_PASSED
Sub-NUMA_appears_to_be_enabled
Huge_Pages_Size_is_50000_Check_PASSED
IOMMU_Passthrough_ENABLED
1077_GiB_Host_Memory_Meets_Minimum_Recommended_96_GiB_Check_PASSED
Device_0000:b0:00.0_Driver_Binding_Check_PASSED
Device_0000:c0:00.0_Driver_Binding_Check_PASSED
Device_0000:b0:00.0_PCI_tree_link_check_PASSED
Device_0000:c0:00.0_PCI_tree_link_check_PASSED
Device_0000:b0:00.0_ACS_ON_PORT_0000:a9:04.0_IS_ENABLED
Device_0000:c0:00.0_ACS_ON_PORT_0000:b9:04.0_IS_ENABLED
Device_0000:b0:00.0_Temperature_30C_Check_PASSED
Device_0000:c0:00.0_Temperature_29C_Check_PASSED
Device_Temperature_Delta_is_1C_Check_PASSED
Device_0000:b0:00.0_Memory_Temperature_is_30C
Device_0000:c0:00.0_Memory_Temperature_is_30C
Device_0_CLPeak_Test_PASSED
Device_1_CLPeak_Test_PASSED
Device_0_IFWI_"BMG_21.1160"
Device_1_IFWI_"BMG_21.1160"
CInfo_Check_Found_2_of_2_Devs_PASSED
No_Failures_Detected
*****END_DEVICE_SUMMARY*****
```

The test summary ends with an overall statement indicating if all tests passed, one or more tests failed, or if warnings or execution failures occurred. Above that are the individual results per device for each check.

The checks performed and information collected by the script are as follows:

- Baseboard information
- Driver module information

- lspci tree and device details
- dmidecode
- BAR allocation
- OS environment
- Device PCIe* link speed and width
- Link speed and width for each node in the accelerator bus path
- OS kernel command line
- Driver bindings
- NUMA settings
- Huge pages settings
- IOMMU passthrough setting
- Host memory quantity check
- GPU Device-to-device temperature delta
- GPU core frequency
- Accelerator firmware version capture
- System journal since current and previous boot
- Workload test and result
- clinfo device detection
- BMC sensors, if available
- Host sensors, if available
- Installed package list

After completion of the device checks and information collection, the script generates a `debugLogs` folder. If the script is launched directly, the logs will be saved in the `InfoCollect.sh` script directory in the format `debugLogs-<year>-<month>-<day>_<H:M:S>`, otherwise, the logs will be in the wrapper `./logs` directory. This folder contains the following log files:

- **DiagInfo:** Includes a test summary with the results for each test and device and host system information, including:
 - Baseboard details
 - Driver module information
 - lspci output
 - dmidecode output
 - Kernel command line
 - Loaded driver modules
 - BMC SEL dump and BMC sensor readings
- **dmesg:** Contains the system dmesg output.
- **lastBootJournalctl:** Contains the system journal from boot -1.

- **journalctl**: Contains the system journal from the current boot.
- **DiagInfoExtra**: Includes captures from various OS file systems under `/bus/pci/devices` and `/proc/`, including:
 - Cpuinfo
 - Interrupts
 - Iomem
 - Meminfo
 - Modules
 - Mtrr
 - Version

Along with the test results, each log file contains an index at the end. These indices provide searchable names for each test and data collection area. Searching for an index name jumps to the corresponding section in a log file.

These logs can be used to assist in debugging and troubleshooting system issues or can be sent to an Intel representative for analysis.

4.16 HTML Summary Report

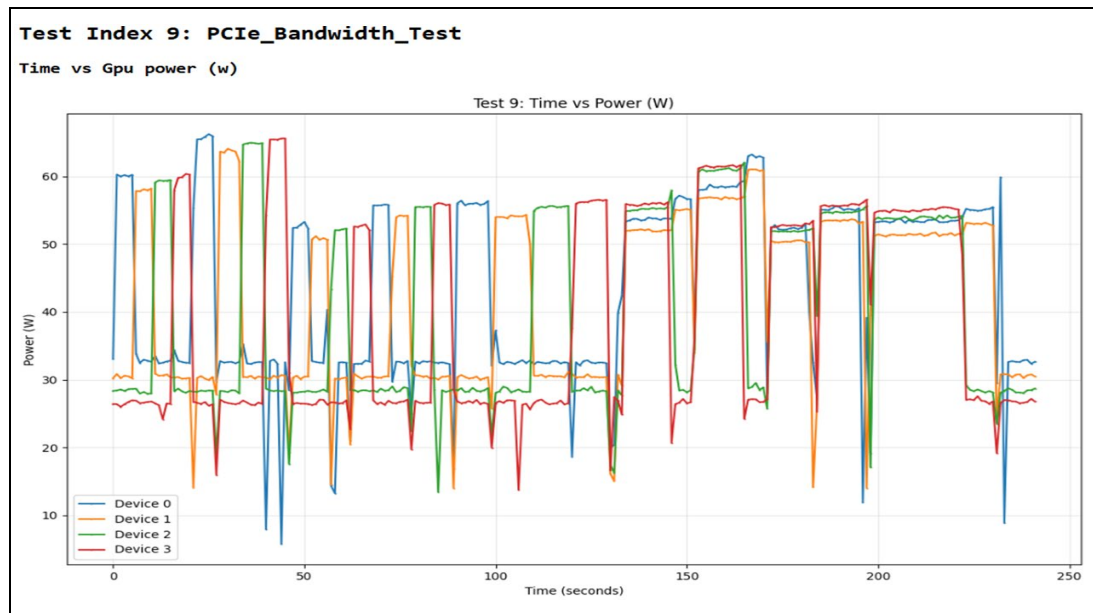
When the **Collect Logs** option is selected from the Wrapper **Tests Menu** or called from the command line, the VTS generates an HTML summary report. The report contains a table of all the test logs found under the VTS logs folder and their results:

Figure 4-2. Summary Report Example

Intel Verification Test Suite Results Summary				
Test Index	Test Name	Test Result	Log File	Date
1	GPU_Health_Check	Pass	GPU_Health_Check_execution_20251219_002506_testPass.log	2025-12-19 00:25:06
2	GPU_Environment_and_Device_Check	Pass	GPU_Environment_and_Device_Check_execution_20251219_002527_testPass.log	2025-12-19 00:25:27
3	PCIe_Bandwidth_Test	Pass	PCIe_Bandwidth_Test_execution_20251219_002808_testPass.log	2025-12-19 00:28:08
4	PCIe_Margin_Test	Pass	PCIe_Margin_Test_execution_20251219_003029_testPass.log	2025-12-19 00:30:29
5	PCIe_Reset_Test_testonly	Fail	PCIe_Reset_Test_testonly_execution_20251219_004201_testFail.log	2025-12-19 00:42:01
6	PCIe_Reset_Test_testonly	Fail	PCIe_Reset_Test_testonly_execution_20251219_004222_testFail.log	2025-12-19 00:42:22
7	PCIe_Reset_Test_testonly	Fail	PCIe_Reset_Test_testonly_execution_20251219_004243_testFail.log	2025-12-19 00:42:43
8	Memory_Bandwidth_Test	Pass	Memory_Bandwidth_Test_execution_20251219_004304_testPass.log	2025-12-19 00:43:04
9	Memory_Stress_Test	Pass	Memory_Stress_Test_execution_20251219_004326_testPass.log	2025-12-19 00:43:26
10	Power_and_Thermal_Stress_Test	Pass	Power_and_Thermal_Stress_Test_execution_20251219_004456_testPass.log	2025-12-19 00:44:56
11	Excursion_Design_Power_Stress_Test	Fail	Excursion_Design_Power_Stress_Test_execution_20251219_005112_testFail.log	2025-12-19 00:51:12
12	AI_VLLM_Test	Fail	AI_VLLM_Test_execution_20251219_005728_testFail.log	2025-12-19 00:57:28

The column “Test Name” contains hyperlinks to the details section for each test which will have plots for GPU’s Power, Temperature, and Frequency vs Time, as shown in the following figure:

Figure 4-3. Test Details Section Example



The “Log File” column also contains hyperlinks to the specific test log where details regarding why a test is passing or failing can be found.

The VTS logs zip file must be unzipped before plots can be visualized, and the hyperlinks in the table can be used.

5 *Issues and Workarounds*

This section contains known issues and workarounds found during testing and development. These items are relevant as of this document and VTS version release. Some of these issues may be resolved in future updates.

5.1 **IOMMU Can Cause ACS to Reenable or Fail to Disable**

For peer-to-peer device communication, ACS must be disabled. If IOMMU is enabled on the host system, it may cause ACS to reenable a few seconds after disabling. To work around this, disable IOMMU by adding `iommu=off` to the `GRUB_CMDLINE_LINUX_DEFAULT=""` line found in `/etc/default/grub`. Rebuild the grub boot loader and reboot.

5.2 **Running Some Card Tests While Running XPUM Can Cause a Crash or Bus Fatal Error**

If XPUM is collecting telemetry or generating traffic on a target device during tests that reset the endpoint or disable the target device or device PCIe link, a kernel panic or bus fatal error may occur. Do not collect telemetry while manipulating device states. If running `xpu-cli`, the `xpum` service must be stopped before running such tests. This is expected behavior, as PCIe requires all traffic to be quiesced before manipulating the bus state.

5.3 **Intel® DG Diagnostic Tool Tests Only Run on Devices Located on PCIe Segment 0**

For systems with multiple PCIe segments or domains, such as `0000:0e:00.0` and `0001:0e:00.0` and so on, Intel® DG Diagnostic Tool currently runs only on devices located on segment 0.

5.4 **PCIe Lane Margin Tool (LMT) Can Cause PCIe Fatal Bus Errors and Fails to Finish**

Running PCIe LMT can cause the PCIe links under test to generate fatal bus errors and may require rebooting the host system.

5.5 PCIe BW Test Bi-Directional memory_benchmark May Fail

When measuring PCIe bandwidth using the memory_benchmark_L0 test in bidirectional mode, a lower-than-expected BW may be observed. This is currently under investigation.

5.6 Power, Thermal, and Excursion Design Power Stress (EDP) Tests May Not Work with Intel® Arc™ Pro B70 GPU Cards

Power, Thermal, and EDP Stress Tests might fail when running on systems with Intel® Arc™ Pro B70 GPU cards with the following error: "MESA: warning: Driver does not support the 0xe223 PCI ID." To fix this, install the latest MESA drivers as follows:

```
sudo add-apt-repository ppa:kisak/kisak-mesa
sudo apt update && sudo apt upgrade
```

5.7 GPU Environment and Device Check Memory Bandwidth Tests May Fail due to Lower Expected Memory Bandwidth

For systems with two or more PCIe* accelerators installed, the GPU Environment and Device Check Memory Bandwidth Tests may fail due to low memory bandwidth measurements. This issue is currently under investigation. Normal performance results have been observed on single-accelerator systems.

5.8 Monitoring and CPU stress tests do not work with vLLM benchmark

During the vLLM benchmarking test, the monitoring service is unable to simultaneously collect GPU utilization metrics and execute CPU stress tests through command line input parameters -mt and -cs.

6 *Troubleshooting*

6.1 Live Monitoring Windows Not Showing

6.1.1 X11 Forwarding Not Enabled

If the live monitoring window does not appear, ensure that X11 forwarding was enabled at login time. If you logged in without X11 forwarding, disconnect and reconnect using `ssh -Y` or reconfigure PuTTY and reconnect.

6.1.2 DISPLAY Variable Not Set

Check the DISPLAY variable using `'echo $DISPLAY'`. If it is empty, X11 forwarding is not active. Reconnect with X11 forwarding enabled.

6.1.3 X Server Not Running on Client

Ensure an X server is running on the client system. On Windows*, start VcXsrv or Xming before opening the SSH session.

6.1.4 Manual X11 Test

Run `'xclock'` on the host system. If a clock window appears on the client, X11 forwarding is working. If you see `'Cannot open display'`, X11 forwarding is not configured correctly.

6.1.5 Root Permission and Xauthority Issues

Some monitoring tools run with elevated privileges. In this case, root must have access to the X server. If you see Xauthority or DISPLAY mismatch warnings, log out completely, reconnect with X11 forwarding enabled, and retry.

6.1.6 Mixed SSH Sessions

Avoid mixing SSH sessions with and without X11 forwarding. If this occurs, log out of all sessions and reconnect using `ssh -Y`.

6.1.7 Restricted Environments

In BMC virtual console environment, or any other restricted environments where X11 forwarding is disabled, live monitoring will not be available. The test will still run, but the monitoring window will be skipped.

6.2 vLLM benchmarking

- **Docker not running:** Start Docker using `sudo systemctl start docker`.
- **Docker login failure:** Verify `DOCKER_USER` and `DOCKER_TOKEN`.
- **Port conflict:** Ensure port 8000 is free or change the port.
- **Server not ready:** Inspect `/tmp/vllm-server.log`, reduce model size, or lower TP/DP values.
- **Model memory check failure:** Ensure the model name includes a parameter-size suffix (for example, (14B)).
- **Proxy issues:** Verify proxy variables and confirm Docker reports active proxy settings via `docker info`.